

DATA GOVERNANCE FOUNDATIONS • 60-MINUTE SESSION

Data Quality Governance

Accuracy • Consistency • Completeness
Integration Governance across CRM, Cloud & Apps

One-Hour Session | Hands-on with Databricks

Created by Prachi Kabra

Instructor-led | Level: Intermediate | Duration: 60 minutes

0. Session Plan

This is a tightly scoped 60-minute session. It teaches three measurable quality dimensions, one enforcement mechanism, and one integration control — demonstrated live on Databricks. It is deliberately not a comprehensive treatment; see "What this session does not cover" below.

Run sheet

Time	Segment	Format
0–5 min	Why quality fails silently (the hook)	Story / discussion
5–15 min	The three dimensions as measurable checks	Teach
15–30 min	Enforcing quality in Databricks	Teach + live SQL
30–45 min	Integration governance across platforms	Teach + live demo
45–55 min	Live demo: catching three real defects	Demo
55–60 min	Scorecard, close, and what to do Monday	Wrap

Learning objectives

By the end of 60 minutes, participants can:

- Express accuracy, completeness, and consistency as a query that returns a number.
- Choose between a Delta constraint and a DLT expectation, and explain the three expectation actions.
- Name the top integration risks when combining CRM, cloud, and app data.
- Run a cross-platform reconciliation and interpret the gap.

What this session does NOT cover

Say this out loud at the start — it sets expectations and prevents scope-creep questions from derailing the hour.

- Full identity resolution and fuzzy matching (mentioned, not taught).
- Delta Live Tables pipeline setup and orchestration (expectations shown as code only).
- Lakehouse Monitoring, drift detection, and alert configuration (named, not demoed).
- Building the full metrics/scorecard framework (one table shown, not built).

Prerequisites & setup

- Comfort with SQL; familiarity with bronze/silver/gold layering.
- A Databricks workspace with Unity Catalog enabled and a running SQL warehouse.
- Instructor: pre-create the demo tables in Section 5 before the session — do not spend live minutes on setup.

Instructor note — the one thing to protect

If you fall behind, cut Section 3 depth, not Section 4. Integration governance is the segment most likely to match the audience's real pain and is the least covered elsewhere. Losing it costs more than losing an enforcement example.

1. Why Quality Fails Silently (0–5 min)

Open with stakes, not definitions. Security and documentation failures announce themselves — an auditor asks, a request is denied. Quality failures do not. Bad data flows quietly into a dashboard and someone makes a decision on it.

Use one of these openers

- The unit mismatch: CRM stores amounts in dollars, the app stores cents. Unioned without conversion, revenue is off by 100x for part of the customer base — and every total still "looks like a number".
- The duplicate customer: the same person exists three times across systems, so a campaign mails them thrice and the customer count that goes to the board is inflated.
- The orphan record: orders referencing a customer who no longer exists, quietly dropped by an inner join, so revenue silently under-reports.

The framing line: documentation tells you what a column means, and security tells you who may see it — neither tells you whether the number is right. That is this session.

1.1 Quality vs. quality governance

Data quality is how fit-for-use the data is. **Data quality governance** is the owned, automated, measured loop that keeps it that way: define → enforce → measure → alert → gate. A one-off cleanup is quality work; the loop is governance.

2. Three Dimensions, Three Queries (5–15 min)

The core skill of the hour: turning a vague complaint ("the data is bad") into a query that returns a number you can threshold, trend, and alert on. Teach one query per dimension and move.

Dimension	Question	What "good" looks like
Completeness	Is required data present?	% of required fields populated
Accuracy	Does it match a trusted reference?	Zero variance vs. system of record
Consistency	Do systems agree with each other?	Zero gap across CRM / app / DB

Completeness

```
-- % of active customers that have an email
SELECT ROUND(100.0 * COUNT(email) / COUNT(*), 2) AS pct_complete
FROM dq_demo.silver.dim_customer
WHERE is_active = true;
```

Accuracy (needs a reference)

Accuracy is the hardest dimension because it requires a trusted source of truth — a ledger, a control total, or a business rule. You detect disagreement rather than proving truth.

```
-- Does our revenue agree with the finance ledger?
SELECT s.order_date, SUM(s.amount_cents) AS our_rev,
       l.ledger_rev, SUM(s.amount_cents) - l.ledger_rev AS variance
FROM dq_demo.gold.fct_sales s
JOIN dq_demo.ref.finance_ledger l USING (order_date)
GROUP BY s.order_date, l.ledger_rev
HAVING variance <> 0;
```

Consistency

```
-- Referential: orders pointing at customers that do not exist
SELECT o.order_id
FROM dq_demo.silver.app_order o
LEFT JOIN dq_demo.silver.dim_customer c USING (customer_id)
WHERE c.customer_id IS NULL;
```

The rule to repeat all hour

A check that only prints bad rows cannot be governed. Reduce every check to a number with a timestamp — that is what powers thresholds, trends, alerts, and audit evidence.

3. Enforcing Quality in Databricks (15–30 min)

Detection is not enforcement. Databricks gives you three mechanisms; the design decision is which to use where.

Mechanism	Behaviour	Use for
Delta constraint	Fails the write / transaction	Hard invariants that must never break
DLT expectation	Warns, drops, or fails — and records metrics	Pipeline gates you want to measure
Custom SQL/PySpark	Anything bespoke or cross-system	Accuracy and cross-platform consistency

3.1 Delta constraints — hard guarantees

```
ALTER TABLE dq_demo.silver.app_order
  ADD CONSTRAINT amount_positive CHECK (amount_cents >= 0);

ALTER TABLE dq_demo.silver.app_order
  ALTER COLUMN order_id SET NOT NULL;
-- Any write violating these fails the whole transaction.
```

3.2 DLT expectations — gates that measure

This is the most reusable slide of the session. Each expectation is a boolean rule plus an action, and DLT records pass/fail counts automatically — so you get a quality trend without writing a metrics job.

Expectation	On failure	Use when
expect	Keep the row, record the metric	You want visibility, not blocking
expect_or_drop	Drop the row, keep loading	Bad rows must not reach silver
expect_or_fail	Stop the pipeline	The rule is non-negotiable

```
import dlt

@dlt.table(name="silver_app_order")
@dlt.expect_or_drop("valid_order", "order_id IS NOT NULL AND amount_cents >= 0")
@dlt.expect("has_email", "customer_email IS NOT NULL") # warn only
@dlt.expect_or_fail("known_status",
  "status IN ('placed','shipped','delivered','cancelled')") # stop
def silver_app_order():
  return dlt.read_stream("bronze_app_events").filter("event_type='order'")
```

3.3 Quarantine instead of dropping

Dropping bad rows loses them. Route failures to a quarantine table so nothing disappears silently and someone can correct and reprocess.

```
from pyspark.sql.functions import expr, current_timestamp

RULES = "order_id IS NOT NULL AND amount_cents >= 0"
flagged = spark.read.table("dq_demo.bronze.orders").withColumn("is_valid", expr(RULES))

(flagged.filter("is_valid").drop("is_valid")
  .write.mode("append").saveAsTable("dq_demo.silver.orders"))

(flagged.filter("NOT is_valid"))
```

```
.withColumn("quarantined_ts", current_timestamp())  
.write.mode("append").saveAsTable("dq_demo.quarantine.orders"))
```

Quarantine needs an owner

An unworked quarantine table is just a slower way to lose data. Assign an owner, alert on non-zero counts, and provide a reprocessing path back into silver.

4. Integration Governance (30–45 min)

Each source can be internally clean and still produce garbage when combined. This segment is where most audiences recognise their own pain — give it the full fifteen minutes.

4.1 Where it breaks

Source	Example	Typical failure
CRM	Salesforce, HubSpot	Duplicates, free-text, stale records, dollars
Cloud database	Azure SQL, Postgres	Schema drift, hard deletes, timezone mixups
App / events	Web app, Kafka	Missing or late events, semi-structured JSON, cents

The four controls that address these, in order of payback:

1. Data contracts — validate schema and freshness at the ingestion boundary.
2. Schema-drift handling — evolve safely instead of failing or silently dropping.
3. Deduplication — resolve the same entity across systems to one golden record.
4. Reconciliation — prove independent systems tell the same story.

4.2 Data contracts at the boundary

```
EXPECTED = {"account_id":"string","email":"string",
            "amount_usd":"double","last_modified":"timestamp"}

crm      = spark.read.table("dq_demo.bronze.crm_raw")
actual = {f.name: f.dataType.simpleString() for f in crm.schema.fields}
breach = [c for c in EXPECTED if actual.get(c) != EXPECTED[c]]
if breach:
    raise ValueError(f"CRM contract breach: {breach}")
```

4.3 Schema drift with Auto Loader

Sources change without asking. Auto Loader can add new columns and rescue unexpected fields, making drift visible and recoverable rather than silent.

```
(spark.readStream.format("cloudFiles")
 .option("cloudFiles.format","json")
 .option("cloudFiles.schemaEvolutionMode","addNewColumns")
 .option("cloudFiles.schemaLocation","/Volumes/dq_demo/ops/schema/app")
 .load("/Volumes/dq_demo/landing/app_events/")
 .selectExpr("*","_rescued_data") # nothing is silently dropped
 .writeStream.option("checkpointLocation","/Volumes/dq_demo/ops/ck/app")
 .toTable("dq_demo.bronze.app_events"))
```

4.4 Deduplication to a golden record

```
from pyspark.sql import Window
from pyspark.sql.functions import row_number, col, lower, trim

w = Window.partitionBy(lower(trim(col("email")))).orderBy(col("updated_ts").desc())

(spark.read.table("dq_demo.silver.customer_unified")
 .withColumn("rn", row_number().over(w))
 .filter("rn = 1").drop("rn")
 .write.mode("overwrite").saveAsTable("dq_demo.gold.dim_customer"))
```

4.5 Cross-platform reconciliation

The single highest-value integration control: prove the app and the cloud order database agree before anyone trusts revenue.

```
WITH a AS (SELECT order_date, SUM(amount_cents) rev
            FROM dq_demo.silver.app_order GROUP BY order_date),
     d AS (SELECT order_date, SUM(amount_cents) rev
            FROM dq_demo.silver.db_order GROUP BY order_date)
SELECT order_date, a.rev AS app_rev, d.rev AS db_rev,
       COALESCE(a.rev,0) - COALESCE(d.rev,0) AS gap
FROM a FULL OUTER JOIN d USING (order_date)
WHERE COALESCE(a.rev,0) <> COALESCE(d.rev,0);
```

5. Live Demo — Catching Three Defects (45–55 min)

One compact demo that exercises everything taught. Pre-create these tables before the session; run only the detection queries live so the ten minutes are spent on findings, not typing.

Setup (run before the session)

```
CREATE CATALOG IF NOT EXISTS dq_demo;
CREATE SCHEMA IF NOT EXISTS dq_demo.silver;

-- CRM: amounts in DOLLARS, contains a duplicate customer
CREATE OR REPLACE TABLE dq_demo.silver.crm_customer AS SELECT * FROM VALUES
  (1, 'asha@example.com', 100.00),
  (2, 'LIAM@example.com', 50.00),
  (2, 'liam@example.com', 50.00)
  AS t(customer_id, email, amount_usd);

-- App: amounts in CENTS, contains an orphan order
CREATE OR REPLACE TABLE dq_demo.silver.app_order AS SELECT * FROM VALUES
  (101, 1, 'placed', 10000),
  (102, 2, 'delivered', 5000),
  (103, 3, 'placed', 2000)
  AS t(order_id, customer_id, status, amount_cents);
```

Defect 1 — Uniqueness: the duplicate customer

```
SELECT customer_id, COUNT(*) AS n
FROM dq_demo.silver.crm_customer
GROUP BY customer_id HAVING COUNT(*) > 1;
```

Talk track: the same person, two casings of the same email. An inner join now multiplies rows and inflates the customer count.

Defect 2 — Consistency: the orphan order

```
SELECT o.order_id
FROM dq_demo.silver.app_order o
LEFT JOIN dq_demo.silver.crm_customer c USING (customer_id)
WHERE c.customer_id IS NULL;
```

Talk track: order 103 references a customer who does not exist. An inner join would drop it silently and under-report revenue — the failure mode nobody notices.

Defect 3 — Consistency: the unit mismatch

```
-- CRM is dollars, app is cents. Naive comparison is off by 100x.
SELECT c.customer_id,
       c.amount_usd AS crm_dollars,
       SUM(o.amount_cents) AS app_cents,
       CAST(c.amount_usd * 100 AS BIGINT) AS crm_cents_fixed,
       CAST(c.amount_usd * 100 AS BIGINT) - SUM(o.amount_cents) AS gap
FROM dq_demo.silver.crm_customer c
JOIN dq_demo.silver.app_order o USING (customer_id)
GROUP BY c.customer_id, c.amount_usd;
```

Talk track: this is the defect that survives every schema check — both columns are valid numbers. Only a semantic contract or a reconciliation catches it.

The fix: one golden, conformed table

```
CREATE OR REPLACE TABLE dq_demo.gold.dim_customer AS
WITH ranked AS (
  SELECT customer_id,
         lower(trim(email)) AS email,
         CAST(amount_usd * 100 AS BIGINT) AS amount_cents,
         ROW_NUMBER() OVER (PARTITION BY lower(trim(email))
                             ORDER BY customer_id) AS rn
  FROM dq_demo.silver.crm_customer)
SELECT customer_id, email, amount_cents FROM ranked WHERE rn = 1;
```

Land this point

Three defects, three dimensions: uniqueness, referential consistency, and cross-system consistency. None would be caught by a schema check — every value was individually valid. That is why quality needs its own controls.

6. Scorecard & Close (55–60 min)

6.1 Make it a number, make it visible

Every check from this session ends as a row in one metrics table. That table is what turns quality from opinion into a trend — and doubles as audit evidence.

```
CREATE TABLE IF NOT EXISTS dq_demo.governance.dq_results (
  run_ts TIMESTAMP, dataset STRING, rule_name STRING, dimension STRING,
  records_checked BIGINT, records_failed BIGINT,
  score_pct DOUBLE, status STRING); -- PASS | WARN | FAIL
```

Aggregated, it becomes the scorecard the business actually looks at:

Dataset	Dimension	Score	Status
gold.dim_customer	Uniqueness	100%	PASS
silver.app_order	Consistency (orphans)	66.7%	FAIL
gold.fct_sales	Accuracy (vs ledger)	100%	PASS

6.2 The closing idea

The loop, one more time: **define** → **enforce** → **measure** → **alert** → **gate**. The last step is what separates governance from cleaning — a failed check should block the gold publish so bad data never reaches a consumer.

6.3 What to do Monday

Close by asking each participant to commit out loud to one action. Adoption, not comprehension, is where governance training usually fails.

- Pick one dataset you own and write a completeness check for its most important column.
- Add one Delta CHECK constraint to a table where a bad value would be unrecoverable.
- Run a reconciliation between two systems that are supposed to agree — and look at the gap.
- Find out who owns the worst-quality dataset you depend on.

Key takeaway

Accuracy, completeness, and consistency are only trustworthy when they are measured continuously and enforced automatically — and integration is exactly where they are won or lost. On Databricks: contracts and Auto Loader guard the boundary, expectations and quarantine guard the pipeline, reconciliation guards cross-platform truth.

Appendix A — One-Page Recap

Hand this out at the end.

- Turn every quality complaint into a query that returns a number.
- Completeness = is it there. Accuracy = does it match a trusted reference. Consistency = do systems agree.
- Delta constraints for hard invariants; DLT expectations for measured gates.
- expect = warn, expect_or_drop = filter, expect_or_fail = stop.
- Quarantine failures with an owner — never drop silently.
- Contract-check every source at the ingestion boundary.
- Use Auto Loader schema evolution so drift is visible, not silent.
- Standardize keys and units before you join across systems.
- Reconcile CRM / app / cloud DB on a schedule and alert on the gap.
- Log every check to a metrics table; gate the gold publish on it.

Appendix B — If You Have More Time

Natural extensions if this session grows to half a day or full day:

- The remaining dimensions: validity and timeliness, with freshness SLAs.
- Full identity resolution including fuzzy matching and survivorship rules.
- Building the metrics framework and a reusable logging helper.
- Databricks Lakehouse Monitoring for automatic drift and anomaly detection.
- A complete multi-source worked example across all medallion layers.
- Alerting, on-call routing, and quality SLAs with domain owners.

— End of Session —

Created by Prachi Kabra • Data Governance Foundations